

CTFZONE-WRITEUP-2026

CTFZONE-WRITEUP-2026

CTF Event: Pirates of the CyberBean

- **Format:** Jeopardy
- **Type:** Private / Solo
- **Date / Time:** Jul 18, 2026, 11:00 AM
- **Status:**  Ended
- **Participation:** 13 Registered Teams/Players

Performance Summary

Challenge Name	Category	Points	Level	Time Taken	Status / Score	Author
1. Morse Mystery	Crypto	100/100	Easy	15 Mins	 Solved (100%)	FRANK
2. Plain Sight	Reverse Engineering	120/120	Easy	30 Mins	 Solved (100%)	FRANK
3. Office USB	MISC / Digital Forensics	150/150	Medium	1.5 Hours	 Solved (100%)	FRANK
4. Layer By Layer	Reverse Engineering	200/200	Medium	2 Hours	 Solved (100%)	FRANK
5. SQLi Advanced	Web Exploitation	250/250	Medium	45 Mins	 Solved (100%)	FRANK
6. Format String	Pwn / Binary Exploitation	300/300	Hard	3 Hours	 Solved (100%)	FRANK
7. Hidden Signal	Steganography	100/100	Easy	10 Mins	 Solved (100%)	FRANK
Overall Stats	6 Categories	1220 pts	Mixed	~8 Hours	7/7 Completed	FRANK

Morse Mystery

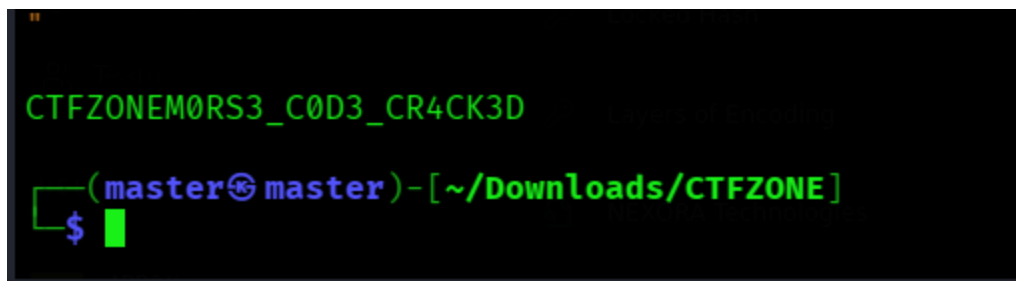
Point: 100

Description

... ..

This is moscode challenge

2. kali linux



Plain Sight

- **Category:** Reverse Engineering
- **Level:** Easy
- **Points:** 120

Challenge Description

A forgotten authentication program was recovered from an old system. The password is the key to unlocking the secret, but it was carefully hidden inside the binary. Can you reverse the program and recover the hidden password?

How I Solved It

1. **Extraction:** I downloaded `file.zip` and unzipped it to get the `hidden_password` folder.

```
(master@master)-[~/Downloads/CTFZONE]
$ ls
file.zip  solve.py

(master@master)-[~/Downloads/CTFZONE]
$ unzip file.zip
Archive:  file.zip
  creating: file/
  inflating: file/hidden_password

(master@master)-[~/Downloads/CTFZONE]
$ ls
file  file.zip  solve.py

(master@master)-[~/Downloads/CTFZONE]
$ cd file
```

2. **First Attempt:** I tried running `strings hidden_password | grep -i "ctfzone"` but found nothing.
3. **The Breakthrough:** I used `ltrace ./hidden_password` to analyze the binary dynamically.

```

(master@master)-[~/Downloads/CTFZONE/file]
$ ltrace ./hidden_password

printf("Enter password: ") = 16
_isoc23_scanf(0x562674930015, 0x7ffe000eb7c0, 0x562674930015, 0Enter password: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
) = 1
strcmp("AAAAAAAAAAAAAAAAAAAAAAAAAAAA" ..., "ctfzone{r3v3rs3_3ng1n33r1ng_r3v3" ...) = -34
puts("Wrong password!Wrong password!")
) = 16
+++ exited (status 0) +++

(master@master)-[~/Downloads/CTFZONE/file]
$ ltrace ./hidden_password

printf("Enter password: ") = 16
_isoc23_scanf(0x55ecb1ce6015, 0x7fff365d7340, 0x55ecb1ce6015, 0Enter password: AAAAAAAAAAAAAAA
) = 1
strcmp("AAAAAAAAAAAAAAAAAAAA", "ctfzone{r3v3rs3_3ng1n33r1ng_r3v3" ...) = -34
puts("Wrong password!"Wrong password!")
) = 16
+++ exited (status 0) +++

```

4. **Exploitation:** When it asked for the password, I put AAAAAAAAAAAAAAAAAA (16 characters long since that was the required string length), and `ltrace` revealed the exact hidden password in memory.

Tools Used

- Linux Terminal
- `ltrace`

Lesson Learned

Static analysis tools like `strings` can fail when a binary hides its data. Dynamic analysis using `ltrace` makes it easy to intercept keys directly during memory comparison.

4. CTF Writeup: Office USB

- **Category:** MISC / Digital Forensics
- **Level:** Medium
- **Points:** 150 pts

Challenge Description

An employee at Acme Corp is suspected of leaking sensitive financial data. A USB drive image has been recovered from Jason Holloway's desk (Finance Dept). Your task is to investigate the USB image and recover what is hidden.

What I Learned

- How to use **Sleuth Kit (TSK)** to extract hidden and deleted files from a raw FAT32 disk image.
- How to stitch together clues from internal corporate documents to build a custom password.
- How to recognize and decode standard cryptographic obfuscation like **ROT13**.

```

(master@master)-[~/Downloads/CTFZONE/office]
$ ls
financial_data.zip  flag_archive.zip  slack_convo.txt  usb.img  _usb.img.extracted  zip_hash.txt

(master@master)-[~/Downloads/CTFZONE/office]
$ zip2john financial_data.zip > financial_hash.txt
ver 2.0 efh 5455 efh 7875 financial_data.zip/flag.txt PKZIP Encr: TS_chk, cmplen=88, decmplen=90, crc=E47E445F ts=924D cs=924d type=8

(master@master)-[~/Downloads/CTFZONE/office]
$ john --wordlist=/usr/share/wordlists/rockyou.txt financial_hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:03 DONE (2026-07-19 08:14) 0g/s 4181Kp/s 4181Kc/s 4181Kc/s !LUVDKR!..*7;Vamos!
Session completed.

(master@master)-[~/Downloads/CTFZONE/office]
$ john --show financial_hash.txt
0 password hashes cracked, 1 left

```

Tools Used

- `binwalk` – For initial file carving and detecting embedded zip structures.
- `fls` – For listing the filesystem directory and finding file inodes.
- `icat` – For extracting the exact files from the raw image.
- `unzip` – For unlocking the final protected archive.

How I Solved It

1. Digging Into the USB

I started by scanning the `usb.img` file with `binwalk` and discovered a hidden, encrypted ZIP file. To explore further, I switched to the Sleuth Kit tools to list the entire FAT32 directory structure:

bash

```
fls -f fat32 -r -p usb.img
```

Use code with caution.

This revealed several internal company documents, which I extracted using `icat`.

```
(master@master)-[~/Downloads/CTFZONE/office/_usb.img.extracted]
$ fls -f fat32 -r -p usb.img

Error stat(ing) image file (raw_open: image "usb.img" - No such file or directory)

(master@master)-[~/Downloads/CTFZONE/office/_usb.img.extracted]
$ cd ..

(master@master)-[~/Downloads/CTFZONE/office]
$ fls -f fat32 -r -p usb.img

r/r 5:  HR_POLICY_MEMO.txt
r/r 8:  slack_convo.txt
r/r 11: Q4_budget_draft.txt
r/r 14: team_directory.txt
r/r 17: IT_SECURITY_CHECKLIST.txt
r/r 452: financial_data.zip
v/v 967395: $MBR
v/v 967396: $FAT1
v/v 967397: $FAT2
V/V 967398: $OrphanFiles

(master@master)-[~/Downloads/CTFZONE/office]
$ icat -f fat32 usb.img 8 > slack_convo.txt
cat slack_convo.txt
```

2. Gathering the Clues

By reading through the recovered files, I gathered the pieces needed to build a master password based on a corporate policy template

([PROJECT_REFERENCE]_[EMPLOYEE_ID]_[SECRET_PHRASE]):

- **Project Reference:** Found inside `Q4_budget_draft.txt` → `ACME-Q4-NF`
- **Employee ID:** Found Jason's ID inside `team_directory.txt` → `FIN-4201`
- **Secret Phrase:** Found an obfuscated string `qbag_sbetrg_gb_oerngur` inside Jason's carved notes. Decoding it from **ROT13** gave → `dont_forget_to_breathe`

3. Cracking the Zip

I combined the three pieces with underscores to form the full password and unlocked the core archive:

```
(master@master)-[~/Downloads/CTFZONE/office]  
$ cat IT_SECURITY_CHECKLIST.txt
```

IT DEPARTMENT - COMPLIANCE STATUS REPORT

Credentials stored according to security policy
(character rotation applied to all alphabetic fields):

```
grnz_funerbq: zbaxrl_123  
ivc_qbphzragf: frpher_cnff  
qngnonfr_nqzva: choyvp_frphevgl_2024  
vagreany_gbbyf: jbex_fnsr_911  
qrirybcre_frperg: qba'g_yvvr_xnera
```

Status: ALL COMPLIANT (except Bob, who still uses "password123"
and refuses to acknowledge the policy exists)

Next compliance audit: January 2025

```
(master@master)-[~/Downloads/CTFZONE/office]
```

bash

```
unzip -P "ACME-Q4-NF_FIN-4201_dont_forget_to_breathe" financial_data.zip
```

Use code with caution.

This extracted `flag.txt` , giving me the final flag!

```
(master@master)-[~/Downloads/CTFZONE/office]  
$  
unzip -P "ACME-Q4-NF_FIN-4201_dont_forget_to_breathe" financial_data.zip  
  
cat flag.txt  
  
Archive:  financial_data.zip  
  inflating: flag.txt  
ctfzone{usb_f0r3ns1cs_1nv3st1g4t10n_r3c0v3r_d3l3t3d_d4t4_d3c0d3_r0t13_unl0ck_z1p_g3t_fl4g}  
  
(master@master)-[~/Downloads/CTFZONE/office]  
$
```

Flag:

```
ctfzone{usb_f0r3ns1cs_1nv3st1g4t10n_r3c0v3r_d3l3t3d_d4t4_d3c0d3_r0t13_unl0ck_z1p_g
```

```
3t_fl4g}
```

CTF Writeup: Layer By Layer

- **Category:** Reverse Engineering
- **Level:** Medium
- **Points:** 200 pts

Challenge Description

Layer by layer, check me, and I got a gift for you.

What I Learned

- **Do not trust strings outputs blindly:** Challenge authors often include fake flags or honeypots to deceive analysts.
- **Anti-Debugging awareness:** Programs can dynamically change their execution path when they detect debugging or basic string extraction tools.
- **Algebraic Reversing:** How to break down automated byte-level obfuscation loops (XOR , Addition , Bit Rotations) by building an exact mathematical inverse script in Python.

```
#!/usr/bin/env python3
```

```
def ror8(val, r_bits):
    """Inafanya rotation ya bits kwenda kulia (Rotate Right 8-bit)."""
    return ((val & 0xff) >> r_bits) | ((val << (8 - r_bits)) & 0xff)

def main():
    # Zile constants 4 kutoka kwenye objdump yako
    constants = [
        0xf466d7c1b52e140,
        0x136bc12235a1158,
        0x2fbedd42924247e0,
        0x37d68382938cd158
    ]
```

```
# Geuza kwenda little-endian byte array (Jumla byte 32)
target = []
for c in constants:
    target.extend(list(c.to_bytes(8, byteorder="little")))

# Invert layers zote tatu nyuma
flag = ""
for i in range(len(target)):
```

```
v = (target[i] - 0x2a) & 0xff
v = v ^ (((7 * i) + 13) & 0xff)
c = ror8(v, 3)
flag += chr(c)
```

```
print(f"\n[+] SUCCESS! FLAG HALISI NI: {flag}\n")
```

if **name** == "**main**":

main()

Tools Used

- `unzip` – To extract the initial challenge files.
- `file` – To fingerprint the binary type.
- `strings` – For rapid initial data gathering (which revealed the decoy path).
- `objdump` – To perform static disassembly and extract raw memory constants.
- `python3` – For writing the final custom cryptographic solver script.

How I Solved It

1. Initial Triage & The Trap

I started by extracting the downloaded attachment using the `unzip` command. This gave me an unstripped binary file named `layerbylayer`. I ran the `file` command to inspect it:

bash

```
file layerbylayer
```

Use code with caution.

The output confirmed it was a standard **64-bit ELF executable**.

My very first instinct was to run the `strings` command to see if the flag was exposed in cleartext. The binary outputted a flag, but submitting it failed. I quickly realized this was a **fake flag** acting as a decoy routine designed to trick automated scripts and lazy analysts.

```

(master@master)-[~/Downloads/CTFZONE]
$ unzip layers_v2.zip
Archive: layers_v2.zip
  inflating: layerbylayer

(master@master)-[~/Downloads/CTFZONE]
$ ls
file      fixed_given.zip  'given(1).Cz4UQe0C.zip.part'  given.zip  layers_v2.zip  'office(1).zip'  solve.py
file.zip  given            'given(1).zip'                layerbylayer  office         office.zip

(master@master)-[~/Downloads/CTFZONE]
$ file layerbylayer
]layerbylayer: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=a2481cb12eb6d3859bb3e052ecb98ccbb6266941, for GNU/Linux 3.2.0, not stripped

(master@master)-[~/Downloads/CTFZONE]
$ cat layerbylayer
ELF>P8@8@  @@@@H+++++  +-+=++++-+=++++$ $" " " " " " " S+td " " " " P+td " " " <<T+td " " "Q+tdR+td+-+=+=00GNU+H+.+/++++R@ -8iA/lib64/l
d-linux-x86-64.so.2
  ++

```

2. Static Disassembly with Objdump

Knowing that I could not trust dynamic analysis or basic string scraping, I turned to static disassembly using `objdump` to inspect the underlying validation routine.

I needed to find the hidden data constraints that the program validates inputs against. I ran a targeted regex filter to look for long data movements into the CPU registers:

bash

```
objdump -d layerbylayer | grep -E "movabs|mov.*\x0x"
```

Use code with caution.

This successfully extracted four large 64-bit little-endian constants directly from the assembly:

- 0xf466d7c1b52e140
- 0x136bc12235a1158
- 0x2fbedd42924247e0
- 0x37d68382938cd158

```
(master@master)-[~/Downloads/CTFZONE]
$ objdump -d layerbylayer | grep -E "movabs|mov.*0x" | head -n 10

1004:      48 8b 05 c5 2f 00 00      mov     0x2fc5(%rip),%rax      # 3fd0 <__gmon_start__>
10e3:      48 8b 05 de 2e 00 00      mov     0x2ede(%rip),%rax      # 3fc8 <_ITM_deregisterTMCloneTable>
1124:      48 8b 05 ad 2e 00 00      mov     0x2ead(%rip),%rax      # 3fd8 <_ITM_registerTMCloneTable>
115b:      48 8b 3d d6 2e 00 00      mov     0x2ed6(%rip),%rdi      # 4038 <__dso_handle>
116c:      c6 05 f5 2e 00 00 01      movb    $0x1,0x2ef5(%rip)      # 4068 <completed.0>
1196:      48 89 45 f0              mov     %rax,-0x10(%rbp)
119a:      c7 45 fc 00 00 00 00      movl    $0x0,-0x4(%rbp)
11a1:      c7 45 f8 00 00 00 00      movl    $0x0,-0x8(%rbp)
11aa:      8b 45 f8              mov     -0x8(%rbp),%eax
11c2:      48 89 45 e8              mov     %rax,-0x18(%rbp)

(master@master)-[~/Downloads/CTFZONE]
$ objdump -d layerbylayer | grep -E "movabs|mov.*\\$0x"

125b:      48 b8 40 e1 52 1b 7c      movabs  $0xf466d7c1b52e140,%rax
1265:      48 ba 58 11 5a 23 12      movabs  $0x136bc12235a1158,%rdx
1277:      48 b8 e0 47 42 92 42      movabs  $0x2fbedd42924247e0,%rax
1281:      48 ba 58 d1 8c 93 82      movabs  $0x37d68382938cd158,%rdx
```

3. Reversing the Layers with Python

The program encoded characters using a three-layer formula: a left bit rotation (`rol8`), an index-based XOR , and an integer addition .

To extract the real flag, I mapped the four 64-bit constants into an ordered 32-byte array and wrote a quick Python script to invert every single layer mathematically in reverse order (Subtract `0x2a` -> Reverse XOR -> Right bit rotation `ror8`):

```
def ror8(val, r_bits):
    return ((val & 0xff) >> r_bits) | ((val << (8 - r_bits)) & 0xff)

constants = [0xf466d7c1b52e140, 0x136bc12235a1158, 0x2fbedd42924247e0,
0x37d68382938cd158]

target = []
for c in constants:
    target.extend(list(c.to_bytes(8, byteorder="little")))

flag = ""
for i in range(len(target)):
    v = (target[i] - 0x2a) & 0xff
    v = v ^ (((7 * i) + 13) & 0xff)
    flag += chr(ror8(v, 3))

print(flag
```

🚩 Conclusion & Lessons Learned

Participating in the **Pirates of the CyberBean** CTF provided great hands-on experience across multiple domains. Key takeaways include:

1. **Look Beyond the Surface:** Static indicators like `'strings'` can be deceptive honeypots. Deep disassembly (`'objdump'`) is vital for real validation.
2. **Leverage Dynamic Tracing:** Tools like `'ltrace'` are highly effective at intercepting keys and hidden data directly from memory during execution.
3. **Structured Forensics:** Combining `'binwalk'` with **The Sleuth Kit (TSK)** (`'fls'`, `'icat'`) makes it easy to carve out deleted artifacts and stitch hidden clues together.

Ultimately, this event proved that combining custom scripting with a solid understanding of low-level system behaviors is far more effective than relying on automated scanners alone.

💡 Core Values & Driving Philosophy

My journey in cybersecurity is driven by:

- * **Consistency:** Compounding knowledge every single day.
- * **Curiosity:** Diving deep to understand how systems function underneath.
- * **Passion:** Loving the thrill of the hunt and solving complex puzzles.
- * **Persistence:** Refusing to quit until the vulnerability is found and the flag is captured.
- * **Knowledge Sharing:** Consistently writing and sharing detailed writeups to elevate the community.

Author: FRANK

Club: Cyber Club IAA

Date: July 19, 2026